

HW-7: Individual Database Project
Pasin Makcharoen 6810545794

Practical Use Option: **C**

live application: <https://school-management-sim-rho.vercel.app/>

github repository:

<https://github.com/Pasin0931/School-Management-Sim>

School Management Simulation

1. Project Scope and Requirements

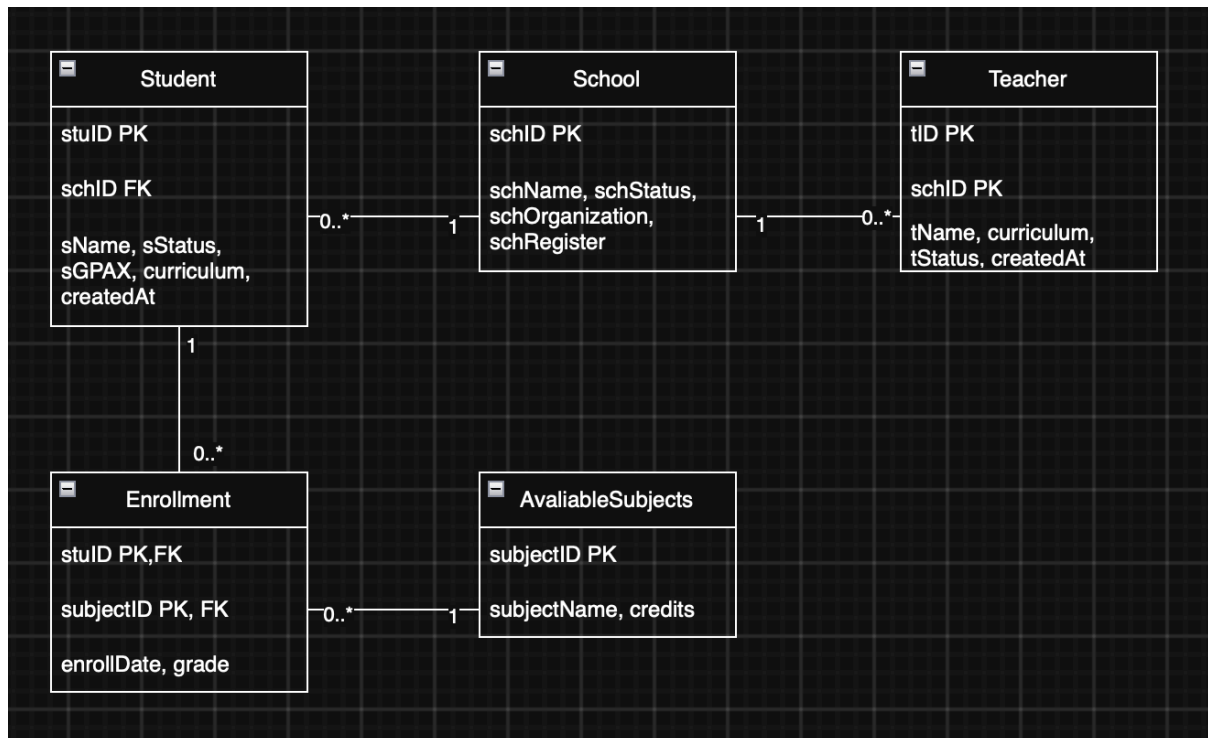
This project is a simplified school management system tracking schools, students, teachers, subjects, and enrollments. It's intended for school administrators who need to register schools, manage student/teacher records, and produce simple enrollment reports. The database must manage: school details (status, ownership type), student records (school, GPAX, curriculum), teacher records (school, specialty, status), available subjects (credits), and enrollments (date, grade).

2. Requirements and Business Rules

- A school can have many students, but each student belongs to exactly one school.
- A school can employ many teachers, but each teacher works at exactly one school.
- A student can enroll in many subjects, and a subject can have many students.
- A student cannot enroll in the same subject twice (each enrollment pair is unique).
- A student's GPAX and curriculum belong to the student, not to any single enrollment.
- A teacher's specialty and status are independent of enrolled students.

3. Top Down Database Design

The UML class diagram has 5 classes: School, Student, Teacher, AvailableSubjects, and the association class Enrollment. School relates 1-to-many to Student and Teacher. Student and AvailableSubjects connect through Enrollment, representing the many-to-many relationship and carrying enrollDate and grade.



4. Functional Dependencies and Keys

Relation	Functional Dependencies
School	schID - schName, schStatus, schOrganization, schRegister
Student	stuID - schID, sName, sStatus, sGPAX, curriculum, createdAt
Teacher	tID - schID, tName, curriculum, tStatus, createdAt
AvailableSubjects	subjectID - subjectName, credits
Enrollment	(stuID, subjectID) - enrollDate, grade

Primary keys: schID, stuID, tID, subjectID, and (stuID, subjectID) for Enrollment. Each uniquely identifies a row since all other attributes in that relation depend fully on it.

5. Normalization

Initial relation: Student_UNF(stuID, sName, sStatus, sGPAX, curriculum, createdAt, schID, schName, schStatus, schOrganization, schRegister)

PK: stuID

Partial Dependencies: None (primary key is a single column)

Full Dependencies:

stuID - sName, sStatus, sGPAX, curriculum, createdAt, schID

schID - schName, schStatus, schOrganization, schRegister

Check for 1NF: satisfied

Check for 2NF: satisfied

Check for 3NF: violated -> schID - school details is a transitive dependency (school details depend on stuID only through schID).

Convert to 3NF: split into Student(stuID PK, schID FK, sName, sStatus, sGPAX, curriculum, createdAt) and School(schID PK, schName, schStatus, schOrganization, schRegister)

6. Database Implementation

Implemented in MySQL 8.4, hosted on Aiven. Includes CREATE TABLE statements with primary/foreign key constraints, and sample data: 5+ rows in each master table (School, Teacher, AvailableSubjects), 10+ rows in Student and Enrollment.

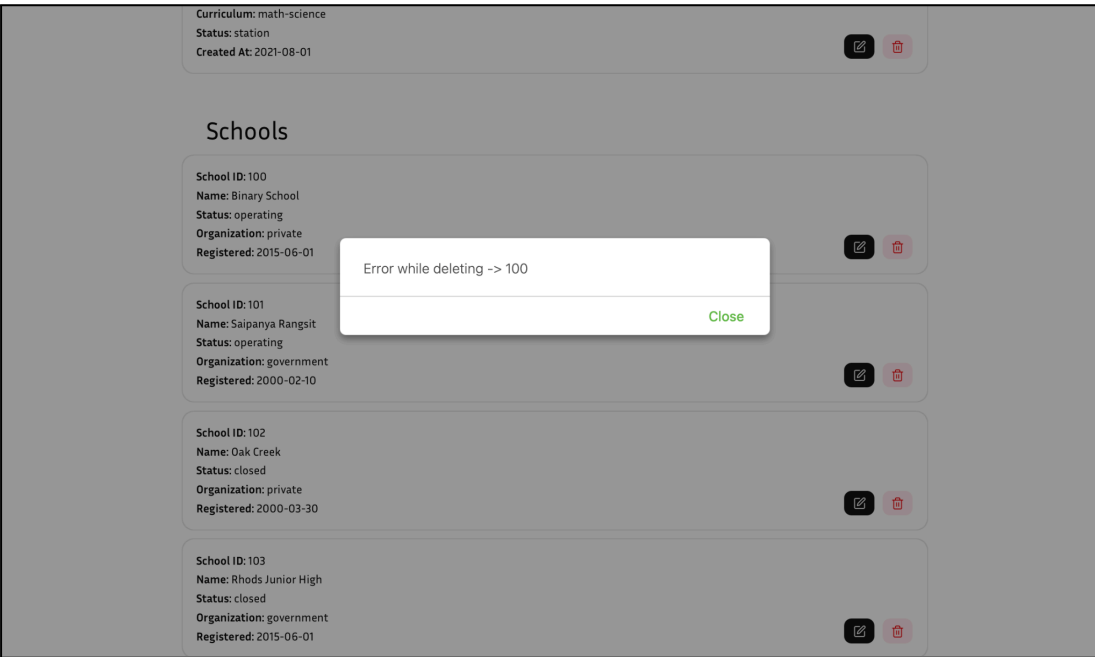
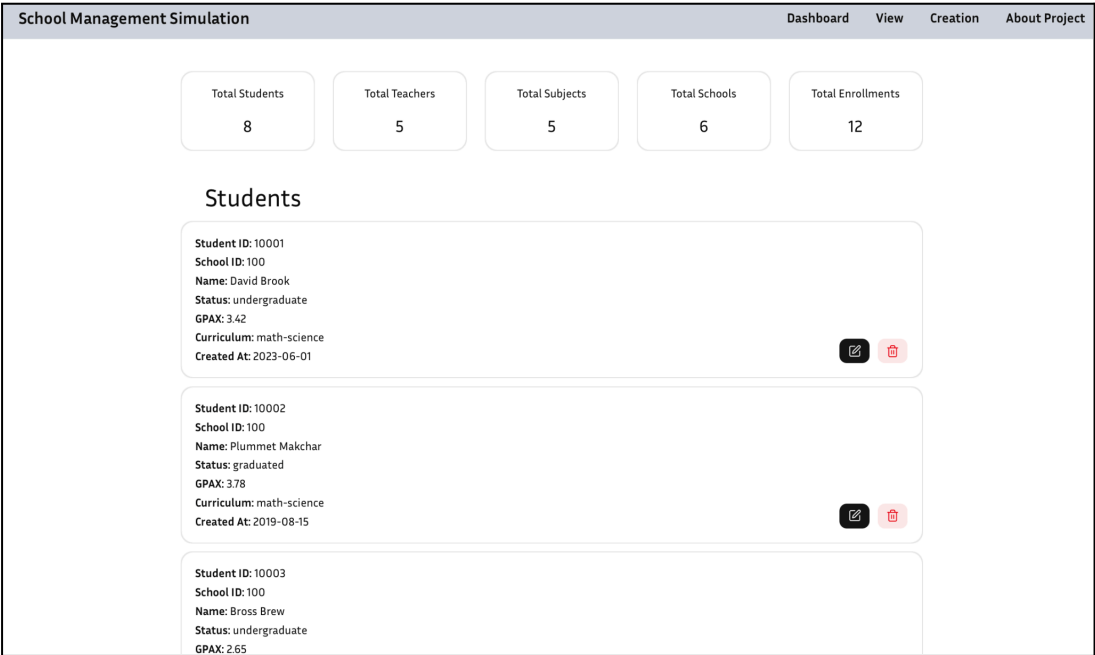
7. Practical Database Use

Option C: a full CRUD web application built with Next.js and MySQL (mysql2, no ORM). Supports adding, viewing, updating, and deleting records for all 5 entities, plus a reporting page with 5 useful SQL queries (join, group by + aggregate, having, subquery, and a business question).

8. Validation and Testing

- Successful insert confirmed via SELECT after adding new records.
- Primary key constraint tested: inserting a duplicate schID correctly failed with a duplicate error.
- Foreign key constraint tested: inserting a Student with a non-existent schID correctly failed with a foreign key error (deleting a School still referenced by a Teacher was also correctly blocked)
- Multi table results confirmed via Query 1, which joins Student and School correctly.

- Results were manually cross checked against raw table data to confirm correctness.



School Management Simulation
Dashboard
View
Creation
About Project

Query Type
Query 5

Query 5: Display Average Grade and Enrollment per Subject

Subject: math	Average Grade: 2.5	Total Students: 4
Subject: science	Average Grade: 3	Total Students: 4
Subject: thai	Average Grade: 2.5	Total Students: 2
Subject: social	Average Grade: 2	Total Students: 1
Subject: english		

School Management Simulation
Dashboard
View
Creation
About Project

Entity Type
student
teacher
school
subject
enrollment
Student name

Status
undergraduate / graduated

GPAX
0.00 - 4.00

Curriculum
e.g. math-science

School ID
School ID

Created At
01 / 01 / 2000

Create

9. Conclusion

This project followed the full database development process for a simple school management system, starting from business rules, designing a UML model, checking normalization (already in 3NF), and implementing the schema in MySQL with proper constraints. A full CRUD web app (Option C) was built on top of it, including five useful SQL reports. Testing confirmed that primary key and foreign key constraints work correctly, and the UML design matched the normalized schema, showing the original design was solid.